- *Publisher/subscriber (pub/sub)*—This model is based on subscriptions. Producers publish events that are sent to all subscribers to be consumed. Events cannot be replayed after being received, so new consumers joining will not be able to get the past events.
- *Event streaming*—In this model, events are written to a log. Producers publish events as they occur, and they are all stored in an ordered fashion. Consumers don't subscribe to them, but they can read from any part of the event stream. In this model, events can be replayed. Clients can join at any time and receive all the past events.

In a basic scenario, consumers receive and process events as they arrive. For specific use cases like pattern matching, they can also process a series of events over a time window. In the event streaming model, consumers have the additional possibility of processing event streams. At the core of event-driven architectures are platforms that can process and route events. For example, RabbitMQ is a common choice to use with the pub/sub model. Apache Kafka is a powerful platform for event stream processing.

The event streaming model is fascinating and growing in popularity, thanks to the many technologies developed in the last few years, allowing you to build real-time data pipelines. It's a complex model, though, that deserves its own book to be taught effectively. In this chapter, I will cover the pub/sub model.

Before we analyze this model in more detail, I'll define some requirements for the Polar Bookshop system, and we'll use them as a means to explore event-driven architectures with the pub/sub model.

10.1.2 Using the pub/sub model